The Journal of Open Source Software

GraphCalc: A Python Package for Computing Graph Invariants in Automated Conjecturing Systems

Randy Davila [1,2]

1 RelationalAI, United States **2** Department of Computational Applied Mathematics & Operations Research,Rice University, United States

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Christoph Junghans

Reviewers:

- @szhorvat
- @aadinoyiibrahim

Submitted: 30 May 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`GraphCalc` is a Python library for computing a broad range of graph-theoretic invariants, purpose-built to support research in combinatorics, network science, and automated reasoning. It offers exact implementations of over 100 functions, spanning classical invariants (e.g., independence number, chromatic number, spectral radius) and a wide array of lesser-known parameters central to contemporary graph theory.

Originally developed as the invariant engine for the automated conjecturing system TxGraffiti, `GraphCalc` has since matured into a general-purpose research tool that facilitates the large-scale construction of structured, high-resolution invariant datasets. These datasets, often organized into tabular "knowledge tables," form the basis for symbolic pattern mining, hypothesis generation, and downstream machine reasoning. For example,

```
>>> import graphcalc as gc
>>> from graphcalc.polytopes.generators import cube_graph, octahedron_graph
>>> graphs = [cube_graph(), octahedron_graph()]
>>> functions = ["order", "size", "spectral_radius", "independence_number"]
>>> gc.compute_knowledge_table(functions, graphs)
   order  size  spectral_radius  independence_number
0      8    12              3.0                    4
1      6    12              4.0                    2
```

While general-purpose graph libraries such as `NetworkX` and `igraph` are widely used for network analysis and algorithmic exploration, they offer limited support for *lesser-known* invariants that often arise in extremal combinatorics and graph theory. `GraphCalc` fills this gap by providing exact implementations of over 100 graph-theoretic parameters—including many that are difficult to compute or entirely unavailable in other Python libraries. This expanded coverage makes it a valuable tool for researchers seeking to analyze symbolic relationships among invariants or generate conjectures from numerical data.

Its design is particularly suited to mathematical exploration on small to medium graphs, where symbolic relationships among invariants are often most visible and meaningful. By enabling fast, programmatic access to both standard and obscure graph parameters, `GraphCalc` lowers the barrier to large-scale experimentation and supports the discovery of new relationships in discrete mathematics. The library includes exact implementations of:

- Fundamental quantities such as *independence number*, *clique number*, and *chromatic number*, Spectral properties related to graph energy, i.e., eigenvalues of adjacency and Laplacian matrices,
- Degree-sequence-based invariants like *residue*, *annihilation number*, and *Slater number*,
- Propagation-based parameters including *zero forcing*, *k-forcing*, and *power domination* variants,

- Domination-type invariants such as *total domination*, *connected domination*, *Roman domination*, *rainbow domination*, and *restrained domination*,
- Structural predicates for classes like claw-free, triangle-free, diamond-free, and bull-free graphs.

Together, these functions cover an expansive and often underrepresented space of graph invariants. Many of these are unavailable in other Python libraries, and all are computed exactly using a mix of solver-based optimization, enumeration, and symbolic methods. As such, `GraphCalc` serves not only as a comprehensive computational toolkit for graph theorists, but as a foundational component in modern systems for automated conjecture generation and symbolic discovery.

Research in graph theory, combinatorics, and automated reasoning frequently relies on the accurate and efficient computation of graph invariants across large collections of discrete structures. Established tools such as `SageMath` (The Sage Developers, 2020), `NetworkX` (Hagberg et al., 2008), and `igraph` (Csárdi & Nepusz, 2006) provide robust support for graph construction, algorithmic exploration, and many standard invariants. `GraphCalc` complements these systems by offering a research-focused interface tailored to conjecture generation and structural analysis—particularly through its extensive support for lesser-known invariants that are often used in extremal graph theory and related areas. This emphasis makes `GraphCalc` especially well-suited for applications in symbolic discovery and mathematical experimentation.

The library implements over 100 exact functions, including many invariants that are unavailable or difficult to compute using existing packages. These span classical quantities, advanced domination-type parameters, spectral metrics, and structural graph properties. In addition to its broad invariant coverage, `GraphCalc` supports batch evaluation over graph collections, returning results as structured "knowledge tables"—tabular datasets suitable for downstream reasoning tasks. This workflow mirrors preprocessing pipelines in machine learning, where raw instances are transformed into rich feature representations.

By integrating high-resolution numerical data with automated conjecturing systems such as `TxGraffiti`, `GraphCalc` enables symbolic pattern discovery at scale—supporting both empirical investigation and the generation of novel mathematical conjectures.

To support this functionality, `GraphCalc` employs a hybrid computational strategy that blends solver-based optimization, exhaustive enumeration, and well-defined algorithms. Many NP-hard invariants—including the independence number, chromatic number, and total domination number—are computed exactly using integer programming models via the PuLP library (Mitchell et al., 2011) and solvers such as the COIN-OR CBC solver, among others. This solver-enhanced foundation allows `GraphCalc` to compute dozens of invariants with provable guarantees, making it an essential tool for both mathematical experimentation and automated discovery.

The utility of `GraphCalc` is most clearly demonstrated through its integration with systems such as `TxGraffiti` and Optimist (Davila, 2025a, 2025b), which generate conjectures by analyzing numerical patterns across large families of graphs. These symbolic patterns emerge directly from the invariant-rich data produced by `GraphCalc`. In this capacity, the library has facilitated the formulation of numerous conjectures—many of which have been rigorously proven as new theorems, while others remain unresolved open problems.

Notably, these conjectures often connect well-studied invariants (e.g., *independence number*, *domination number*, and many generalizations) with less commonly explored parameters (e.g., *residue*, *Slater number*, *zero forcing number*), revealing structural relationships that had previously gone unrecognized. By providing exact and broad invariant coverage—including many parameters absent from `SageMath`, `NetworkX`, `igraph`, and `passagemath-graphs`—`GraphCalc` substantially expands the conjectural search space accessible to modern automated reasoning systems.

Features

GraphCalc offers a robust suite of tools for computing, analyzing, and visualizing graph-theoretic invariants. It combines an intuitive Python interface with solver-enhanced backends and supports both NetworkX graph objects and internal SimpleGraph and related types—making it versatile for everyday use, educational settings, and advanced mathematical experimentation. Key features include:

- **Extensive invariant coverage:** Compute a broad range of exact graph invariants, including classical quantities such as chromatic number, maximum clique, vertex cover, and independence number, as well as structural and degree-based invariants like residue, Slater number, and annihilation number.
- **Domination and forcing variants:** Includes over a dozen domination-type parameters (e.g., total, Roman, double Roman, restrained, outer-connected) and propagation-based parameters such as zero forcing, positive semidefinite zero forcing, and k-power domination. All are computed exactly using integer programming or exhaustive search.
- **Spectral and structural analysis:** Supports spectral computations, including adjacency and Laplacian eigenvalues, spectral radius, and algebraic connectivity, along with Boolean predicates for structural properties such as planarity, claw-freeness, triangle-freeness, and subcubicity.
- **Graph and polytope generators:** Provides built-in generators for classical graphs (derived from NetworkX) and convex 3D polytopes (e.g., tetrahedra, cubes, fullerenes)—useful for visualization, testing, and conjecture exploration.
- **Batch evaluation and knowledge tables:** Enables the evaluation of multiple invariants across entire graph collections using compute_graph_properties_dataframe, which returns results as structured knowledge tables. These tables integrate directly with automated conjecturing systems like TxGraffiti.
- **Visualization and user experience:** Offers built-in rendering for graphs and polytopes, fully type-annotated functions, extensive test coverage, and online documentation designed to support both research and instructional use.

Example Usage

The GraphCalc package supports both single-graph queries and batch evaluation over collections of graphs. Below is a basic example using the *Petersen graph*:

```
>>> import graphcalc as gc
>>> from graphcalc import petersen_graph
>>>
>>> # Create the Petersen graph
>>> G = petersen_graph()
>>> # Compute selected invariants
>>> gc.independence_number(G)
4
>>> gc.residue(G)
3
>>> gc.claw_free(G)
False
```

Relevance to Automated Discovery

Automated mathematical discovery has a rich history, dating back to symbolic logic programs like Wang's *Program II* in the 1950s (Wang, 1960), and advancing significantly with systems such as Fajtlowicz's *Graffiti* (Fajtlowicz, 1988) and DeLaViña's *Graffiti.pc* (DeLaViña, 2005)

DRAFT

in the 1980s and 1990s. These pioneering systems demonstrated that computers could do more than verify known mathematics—they could help *generate* it, particularly by formulating conjectures grounded in patterns among graph invariants. Notably, *Graffiti* included its own embedded module for computing such invariants, a design decision that enabled the system to generate over 60 published conjectures, many appearing in top mathematical journals.

`GraphCalc` continues this lineage. Originally developed as the internal invariant engine for the *TxGraffiti* system, it served for years as a private computational backend before being released as an open-source Python package. This decision was motivated by the growing interest in AI-assisted mathematical reasoning and the desire to make a high-quality, extensible invariant engine available for others to experiment with.

Today, `GraphCalc` powers the latest version of *TxGraffiti* and its agentic counterpart the *Optimist* (Brimkov et al., 2024; Davila, 2025b), which analyze large families of graphs and polytopes to discover symbolic conjectures. To support this process, `GraphCalc` computes a rich spectrum of graph-theoretic quantities—from well-known invariants like *chromatic number* and *independence number* to more specialized parameters such as *positive semidefinite zero forcing*, *Slater number*, and *residue*.

While not optimized for massive-scale network analysis, `GraphCalc` excels in the domain where most mathematical conjectures are formed: *small to medium-sized graphs* that are easily visualized and reasoned about. This design philosophy echoes the foundational principles of Fajtlowicz's original system, which emphasized working with "small but interesting" graphs as fertile ground for discovery. By transforming these structures into structured numerical profiles, `GraphCalc` enables automated systems to detect symbolic patterns and formulate conjectures that are both novel and mathematically meaningful.

Acknowledgements

The authors gratefully acknowledge David Amos and Boris Brimkov for their foundational support during the development of `GraphCalc`. David Amos provided early technical insight and design feedback that shaped the architecture of the package, while Boris Brimkov contributed valuable mathematical guidance in selecting and implementing key graph invariants.

We also thank the many individuals who provided detailed comments on the manuscript and codebase during the revision process. Their feedback significantly improved the quality, clarity, and functionality of both the `GraphCalc` library and this paper.

References

Brimkov, B., Davila, R., Schuerger, H., & Young, M. (2024). On a conjecture of TxGraffiti: Relating zero forcing and vertex covers in graphs. *Discret. Appl. Math.*, *359*, 290–302. https://doi.org/10.1016/j.dam.2024.08.006

Csárdi, G., & Nepusz, T. (2006). The igraph software package for complex network research. *InterJournal Complex Systems*, *1695*. https://doi.org/10.5281/zenodo.3630268

Davila, R. (2025a). *Automated conjecturing in mathematics with TxGraffiti*. https://doi.org/10.48550/arXiv.2409.19379

Davila, R. (2025b). *The optimist: Towards fully automated graph theory research*. https://doi.org/10.48550/arXiv.2411.09158

DeLaViña, E. (2005). *Graffiti.pc*: A Variant of Graffiti. In *Graphs and discovery* (Vol. 69, pp. 71–88). American Mathematical Society. https://doi.org/10.1090/dimacs/069/05

Fajtlowicz, S. (1988). On conjectures of Graffiti. *Discrete Mathematics*, *72*(1), 113–118. https://doi.org/10.1016/0012-365X(88)90199-9

DRAFT

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th python in science conference (SciPy2008)* (pp. 11–15). https://doi.org/10.25080/TCWV9851

Mitchell, S., Dunning, I., & O'Sullivan, M. (2011). PuLP: A linear programming toolkit for python. *Optimization Online*. https://optimization-online.org/2011/09/3178/

The Sage Developers. (2020). *SageMath, the Sage Mathematics Software System (Version 9.1)*. https://doi.org/10.5281/zenodo.4066866

Wang, H. (1960). Toward mechanical mathematics. *IBM Journal of Research and Development*, *4*(1), 2–22. https://doi.org/10.1147/rd.41.0002

DRAFT